

Actuarial & Life-Contingency Code Walkthrough

A maintainer's guide to the "for life" / pay-on-death feature in the DOS source: the integration points that are present, the interface contract they pin down, the recovered mortality-table data — and an honest accounting of the one piece that is still missing.

Read §0 first. This flow is unique among the Per%Sense engines: the DOS source contains the *integration layer* for the actuarial feature but not the *computational core* (the **ACTUARY** unit defining `LifeProb` / `PODValue` / `XPODValue` / `Ondearth`). This document explains the integration points and the table format that *are* present, and clearly flags what is absent.

Scope. The actuarial integration sites in `PRESVALU.pas`, `PVLXSCRN.pas`, `pvltable.pas`; the types and runtime state in `PETYPES.PAS` / `PEDATA.pas`; the interface contract those call sites pin down; and the recovered `MALE.ACT` / `FEMALE.ACT` table data.

Authority & location. DOS line numbers cite `legacy/src_documented/dos_source/` and were recomputed against that tree. The source-availability findings are from `docs/actuarial_oracle_blocked.md` and `docs/actuarial_source_investigation.md`.

Prerequisite. This feature is an overlay on the present-value engine; read `present_value_dos_walkthrough.pdf` first.

Contents

0 How to read this document — the provenance caveat

1 What is present vs absent

1.1 The split · 1.2 The missing ACTUARY unit · 1.3 The recovered table data

2 The interface contract

2.1 The four functions · 2.2 Contingency types · 2.3 Runtime state

3 Integration semantics (the hard-to-guess part)

3.1 `LifeProb` as a weight · 3.2 The JJM special case · 3.3 POD add/subtract symmetry · 3.4 Fancy & table

4 The recovered table-file format

5 What the call sites pin down — and what they don't

6 Appendix: data-flow sketch & file:line index

0 How to read this document — the provenance caveat

The life-contingency feature (the manual's "Actuarial window," used to value an annuity that continues for life or a benefit paid on death) was a real, shipped feature. But it occupies an unusual position in the source we hold, and this document is structured around that fact.

The DOS source is half of this feature. What is **present** is the integration layer — the type system, the runtime state, and the `{%ifdef ACTU}` call sites that show exactly how the actuarial core plugs into present value. What is **absent** is the computational core: a unit named **ACTUARY** containing the functions `LifeProb`, `PODValue`, `XPODValue`, and `Ondeath`. Those are *called* everywhere but *defined* nowhere in the repository — not in any `.pas`, not in a `.dcu`. See `docs/actuarial_oracle_blocked.md` and `docs/actuarial_source_investigation.md`.

Practically, this means:

- **§1–3 (the DOS integration layer and interface contract)** are recoverable and citable — solid ground, with exact `file:line` references into `src_documented/dos_source`.
- **§4 (the table-file format)** describes the recovered `MALE.ACT` / `FEMALE.ACT` data, now embedded in the Go port.
- **§5** states precisely what the scattered call sites do — and do not — pin down about the missing core.

DOS citations look like `PRESVALU.pas:313` and refer to `legacy/src_documented/dos_source/`. Where the Go reconstruction is named, the citation points to `internal/finance/actuarial/` and the companion `actuarial_go_walkthrough.pdf` carries the full Go detail.

1 What is present vs absent

1.1 The split

Layer	Status	Where
Contingency type codes	PRESENT	<code>PETYPES.PAS:253–255</code>
Per-row contingency fields (<code>act0</code> / <code>actn</code>)	PRESENT	<code>PETYPES.PAS:767, 795</code>
Runtime actuarial state & <code>InitActuarial</code>	PRESENT	<code>PEDATA.pas:340–344, 1205</code>
Letter mapping (<code>actchar</code>)	PRESENT	<code>PEDATA.pas:186</code>
<code>{%ifdef ACTU}</code> integration sites	PRESENT	<code>PRESVALU</code> / <code>PVLXSCRN</code> / <code>pvltable</code>
Default table names (' <code>MALE</code> ', ' <code>FEMALE</code> ')	PRESENT	<code>PEDATA.pas:103</code>
<code>LifeProb</code> / <code>PODValue</code> / <code>XPODValue</code> / <code>Ondeath</code> bodies	ABSENT	the ACTUARY unit (not in repo)
<code>ActuarialCalc</code> / <code>ComputeUnknownPOD</code> bodies	ABSENT	called at <code>PRESVALU.pas:1481, 1508, 1550</code> ; defined nowhere
The mortality table <i>data</i>	RECOVERED	<code>MALE.ACT</code> / <code>FEMALE.ACT</code> → Go (§1.3, §4)

1.2 The missing ACTUARY unit

The unit is named explicitly — but commented out — in the `uses` clauses of both consumers: `//{%ifdef ACTU} ,ACTUARY {%endif}` at `PRESVALU.pas:58` and `pvltable.pas:38`. A direct `grep` confirms every occurrence of `LifeProb`, `PODValue`, `XPODValue` and `Ondeath` in `dos_source` is a *call*, never a definition. The build flag `ACTU` is never `{%define}`d in the snapshot, so all integration blocks are dead in *this* build — though they were live in the shipped actuarial build.

1.3 The recovered table data

The source carries the default table names `actuarialfilename: ('MALE', 'FEMALE') PEDATA.pas:103`, backed by an array[1..2] of `str8` field `PETYPES.PAS:589` — the 1988 HHS male/female tables. The *data* of those tables (but not the engine that reads them) has since been **recovered** from a copy of the original software: `MALE.ACT` and `FEMALE.ACT` are parsed into the Go port at `internal/finance/actuarial/persense1988.go` (`Persense1988MaleQx / Persense1988FemaleQx`) and served to the web frontend as the `PERSENSE_1988_*` tables. This closes the *data* half of the blocker; the *engine* half (the missing unit) remains open.

2 The interface contract

Even without the core, the call sites pin down the signatures and semantics the core must satisfy — the hard-to-guess part, and the part the Go port reconstructs against.

2.1 The four functions

```
function LifeProb(date: daterec; contingency: byte): real;
{ P(the contingency holds at `date`), given the lives' DOBs and the
  valuation date actu_now. NOT_CONTINGENT -> 1.0. Used as a
  MULTIPLICATIVE WEIGHT on each cash flow. }

function PODValue(asof: daterec; rate: real): real;
{ PV at `asof`, at fixed `rate`, of the death benefit `pod` paid at
  the uncertain moment of death over the relevant life(s). }

function XPODValue: real;
{ As PODValue, but discounted through the variable-rate schedule to
  d^.xasof -- the PVLfancy path; reads globals, takes no args. }

function Ondearth: {string} ... // the "On death" table label/row helper
```

(Signatures per `docs/actuarial_source_investigation.md`, corroborated by the call sites in §3. `Ondearth` appears as a label in the table writer at `pvltable.pas:685`.)

2.2 Contingency types

The contingency constants `PETYPES.PAS:253–255` and their screen letters `PEDATA.pas:186`. Each lump-sum row carries `act0 PETYPES.PAS:767`; each periodic row carries `actn PETYPES.PAS:795`.

Code	Const	Letter	Condition (pays if...)
0	NOT_CONTINGENT	N	always (no contingency)
1	LIVING	L	life #1 is alive
2	DEAD	D	life #1 is dead
3	ONLY_1_LIVING	1	#1 alive AND #2 dead
4	ONLY_2_LIVING	2	#2 alive AND #1 dead
5	EITHER_LIVING	E	at least one alive (last-survivor)
6	BOTH_LIVING	B	both alive (joint-life)

2.3 Runtime state

The actuarial inputs live in module globals `PEDATA.pas:340–344`: `actu_now` (valuation date), `termdate`, `dob[1..2]` (dates of birth for up to two lives), `pod` (death benefit entered by the user), `podval` (its present value), and `deathdate`. The master switch `fold_in_life` is declared at `PEDATA.pas:211` (a real boolean under ACTU; a false constant otherwise, `PEDATA.pas:188`). `InitActuarial PEDATA.pas:1205` zeroes these, sets `actu_now := now`, builds `actset` from `actchar`, and runs even in non-ACTU builds (`PEDATA.pas:1292`) because the state is serialized to file regardless.

3 Integration semantics (the hard-to-guess part)

Every integration site is gated on the global boolean `fold_in_life`. The structure is a clean symmetry: the survival weight and the death-benefit term are *added* on the forward pass and *removed* on the backward pass.

3.1 LifeProb as a multiplicative weight

Each discounted cash flow is multiplied by its survival probability. Forward, on a lump value `PRESVALU.pas:313`:

```
val0 := amt0 * exp(r.rate * YearsDif(asof,date));           { discount }
if (fold_in_life) then val0 := val0 * LifeProb(date, a[i]^act0); { survival weight }
```

Backward (value → amount) the weight is folded back in via the same summation `PRESVALU.pas:883–884`; a near-zero probability (the date is beyond the life span) is trapped with the `no_time_with_life` guard `PRESVALU.pas:339–340` rather than producing an infinity. The weighted present value of a cash flow is thus $\text{payment} \cdot e^{-r \cdot \Delta t} \cdot \text{LifeProb}(t, \text{contingency})$ — a discount factor times a survival factor. For periodic streams, switching on a survival weight forces the engine onto the slow exact (period-by-period) summation — the closed-form geometric series cannot carry a per-period weight `PRESVALU.pas:420, 541`. Each period's term is multiplied by `LifeProb(t, b[j]^actn) PRESVALU.pas:427, 549`.

3.2 The JJM special case

The single most non-obvious line. `PRESVALU.pas:551`:

```
if (b[j]^actn in [DEAD,ONLY_1,ONLY_2]) and (DateComp(t,actu_now)<=0) then part:=1;
```

The summation loop terminates when `abs(part) > teeny` fails. For death-conditioned contingencies (`DEAD, ONLY_1, ONLY_2`), `LifeProb` is ≈ 0 before `actu_now` (the person is presumed alive now, so hasn't died yet) — but the paying periods come later. Forcing `part := 1` for those early periods keeps the `part < teeny` test from truncating the loop before it reaches the periods that actually pay. Annotated "JJM 2/28/93" in the source (`PRESVALU.pas:554`). A port that drops this will silently produce a zero value for death-conditioned annuities — which is exactly why the Go port's exact-summation loop deliberately never early-breaks (see the Go walkthrough §5.4).

3.3 POD add/subtract symmetry

The death benefit's present value participates in every evaluation. Forward, it is added to the grand total `PRESVALU.pas:896`:

```
if (fold_in_life) then sumvalue := sumvalue + PODValue(asof, r.rate);
```

It also participates inside the unknown-rate secant loop, evaluated at each trial rate `PRESVALU.pas:921, 1001`. Before a backward solve it is subtracted from the target first — so the residual the solver isolates is purely the cash-flow value `PRESVALU.pas:1084–1086`:

```
if (fold_in_life) then
  if PVLfancy then val := val - XPODValue      { variable rate }
  else          val := val - PODValue(c[1]^asof, c[1]^r.rate); { fixed }
```

Conceptually the POD value is the expectation of the discounted benefit over the death-time density. The exact DOS quadrature lived in the missing unit; the Go reconstruction uses a year-of-death summation with a mid-year point-mass, now confirmed against the real DOS program to the cent (Go walkthrough §6).

3.4 Fancy & table integration

The variable-rate path mirrors the fixed one: LifeProb weights fancy lump values `PVLXSCRN.pas:366` and each fancy periodic term `PVLXSCRN.pas:472`, and the fancy grand total adds the precomputed `podval PVLXSCRN.pas:584`. In the table, `PrintNextPayment pvltable.pas:619` splits each row's value into an "if paid" amount and a survival-weighted amount and recovers the probability as `v / ifpd pvltable.pas:493, 546`; `PrintPOD pvltable.pas:679` emits a distinct on-death line (`podsignal` sentinel month `pvltable.pas:540`, the `Ondearth` label at `pvltable.pas:685`) folded into the table total.

4 The recovered table-file format

The two default tables are flat ASCII files named by `actuarialfilename PEDATA.pas:103`. Their *data* was recovered and is documented in the Go port's `persense1988.go` header. The format:

- **One qx value per line**, indexed by exact integer age, where `qx` = probability of death within the year at that age.
- Values are written in the form `"x.xxxE-3"`, i.e. scaled by 0.001 (so `"0.377E-3" = 0.000377`).
- The basis is named in the sibling distribution header `88B&WM&F.ACT` as **"USA Dept of HHS, 1988."**
- **MALE.ACT** runs age 0..115 and closes at `qx[115] = 1.0`.
- **FEMALE.ACT** begins at age 1 — it has *no age-0 row* — and likewise closes at age 115. The Go port preserves this exactly (`qx[0] = 0` for the female table); a reference age of 0 never occurs in practice, so the omission is immaterial but flagged rather than filled.

From `qx` the engine builds survivors `lx[i+1] = lx[i] · (1 - qx[i])` off a radix, then weights cash flows by conditional survival ratios. The Go port's `NewLifeTableFromQx` and `survival` functions reproduce this; see the Go walkthrough §3.

5 What the call sites pin down — and what they don't

An exhaustive walk through every `{$ifdef ACTU}` site recovers more of the contract than the named functions alone — enough to fully determine the annuity arithmetic, even though it recovers neither the survival formula nor the POD quadrature.

Constraint pinned down	Evidence
Date-targeted solves are forbidden under life contingency. When <code>fold_in_life</code> is on, solving for a lump or periodic date raises <code>no_time_with_life</code> rather than iterating — the survival weight makes value-vs-date non-invertible.	PRESVALU.pas:101, 339–340, 1139–1140
The life annuity is a discrete sum over scheduled payment dates. <code>fold_in_life</code> forces the engine off the closed form onto the exact period-by-period method; each rate-discounted payment is weighted by <code>LifeProb</code> at that date.	PRESVALU.pas:420, 427, 541, 549
LifeProb is a pure multiplicative weight in [0,1]. The table prints the row probability as <code>prob := v / ifpd</code> , where <code>ifpd</code> is the discounted-but-not-weighted value and <code>v = ifpd · LifeProb</code> — so the weight is recoverable as a ratio.	pvltable.pas:493, 546, 583
POD participates in the rate root-find. Inside the unknown-rate secant loop, <code>sum := sum + PODValue(asof, r.rate)</code> — so <code>PODValue</code> must be a smooth function of rate, evaluable at every trial rate, not a precomputed constant.	PRESVALU.pas:921, 1001
The unknown-POD solve is structurally reconstructable. <code>podunk</code> is set when all cash-flow blanks are filled but a target is given; <code>ComputeUnknownPOD</code> then back-solves the benefit. Because a death benefit's PV is linear in <code>pod</code> , the solve reduces to a single divide.	PRESVALU.pas:1461, 1489, 1550

What is not recovered. The mortality table *data* is now recovered (§4), but the **survival formula and intra-year interpolation / age convention**, and the **PODValue death-time quadrature**, lived inside the missing **ACTUARY** unit. There is no `DOB`-entry, age-computation, or table-lookup code anywhere in `dos_source` or `win_source` — `dob[1..2]`, `actu_now`, and `termdate` are only declared and zeroed. The Go port's mid-year POD convention was therefore inferred and then *confirmed by elimination* against the real DOS binary (Go walkthrough §6); it is not readable from the source. Obtaining **ACTUARY.PAS/.dcu** would allow a bit-identical engine oracle — see `docs/actuarial_oracle_blocked.md` and `docs/actuarial_dosbox_oracle_plan.md`.

6 Appendix: data-flow sketch & file:line index

```

FORWARD (build value):
  lump i:      val0 = amt0*e^{r*dYears(asof,date)}
               if fold_in_life: val0 *= LifeProb(date, act0)           PRESVALU.pas:313
  periodic j: (EXACT method forced when fold_in_life)                 PRESVALU.pas:420
               per period t: part = amt*e^{-r*dYears(t,asof)}*cola
                           if fold_in_life: part *= LifeProb(t, actn)   :427/549
                           [JJM guard: DEAD/ONLY_1/ONLY_2 before now -> part:=1] :551
  grand total: if fold_in_life: sumvalue += PODValue(asof, rate)       PRESVALU.pas:896
               (= XPODValue, fancy path)

BACKWARD (isolate the unknown):
  val = sumvalue
  if fold_in_life: val -= PODValue / XPODValue   (peel POD first)       PRESVALU.pas:1084-1086
  solve the unknown row, with LifeProb folded back into Summation       PRESVALU.pas:883-884
  date-targeted solve under life contingency: REJECTED                   PRESVALU.pas:339-340 / 1139-1140

Recovered table data (Go):  MALE.ACT / FEMALE.ACT -> persense1988.go (Persense1988Male/Female)
Reconstructed core (Go):    LifeProb / PODValue / LifeTable             internal/finance/actuarial/

```

Symbol	Reference
Contingency codes	PETYPES.PAS:253–255
Per-row fields (act0 / actn)	PETYPES.PAS:767 / 795
actuarialfilename field / array	PETYPES.PAS:589
actchar letter map / default table names	PEDATA.pas:186 / 103
fold_in_life decl / const-false	PEDATA.pas:211 / 188
Runtime state / InitActuarial	PEDATA.pas:340–344 / 1205
Commented-out uses ACTUARY	PRESVALU.pas:58 · pvltable.pas:38
LifeProb weight (lump fwd)	PRESVALU.pas:313
LifeProb weight (periodic) / JJM guard	PRESVALU.pas:427, 549 / 551
no_time_with_life guard	PRESVALU.pas:101, 339–340
POD add (fwd) / in rate loop	PRESVALU.pas:896 / 921, 1001
POD subtract (bwd) / ComputeUnknownPOD	PRESVALU.pas:1084–1086 / 1550
Fancy weighting / podval add	PVLXSCRN.pas:366, 472 / 584
PrintNextPayment / prob=v/ifpd / PrintPOD / Ondearth	pvltable.pas:619 / 493,546 / 679 / 685
Recovered table data (Go)	internal/finance/actuarial/persense1988.go
Reconstructed core (Go)	internal/finance/actuarial/contingency.go · table.go
Provenance / blocker reports	docs/actuarial_source_investigation.md · actuarial_oracle_blocked.md

Per%Sense DOS → Go port. DOS line numbers cite [legacy/src_documented/dos_source/](#); Go citations are under [internal/finance/actuarial/](#). This feature is an overlay on the present-value engine — see [present_value_dos_walkthrough.pdf](#). Source-availability findings: [docs/actuarial_oracle_blocked.md](#), [docs/actuarial_source_investigation.md](#). The Go reconstruction (validated against the real DOS program to the cent on the recovered tables) is detailed in [code_docs/actuarial_go_walkthrough.pdf](#).